CENTRO UNIVERSITÁRIO DA FUNDAÇÃO EDUCACIONAL GUAXUPÉ – UNIFEG
CURSO DE CIÊNCIA DA COMPUTAÇÃO

[NOME COMPLETO – INTEGRANTE 1]
[NOME COMPLETO – INTEGRANTE 2]
[NOME COMPLETO – INTEGRANTE 3]

SEGURANÇA DE SISTEMAS DE INTELIGÊNCIA ARTIFICIAL

Evasão, envenenamento, prompt injection e cadeia de suprimento: um experimento ético, reprodutível e local

Divisão de autoria (conforme checklist de entrega):
Integrante 1 – [preencher: p. ex., experimentos de evasão e envenenamento (seções 3.2, 3.3 e 4.1–4.2); revisão final];
Integrante 2 – [preencher: p. ex., RAG local e prompt injection (seções 3.4 e 4.3); slides];
Integrante 3 – [preencher: p. ex., fundamentação, cadeia de suprimento e referências (seções 2, 3.5 e 4.4); ABNT].

Tema 05 – Disciplina: [nome da disciplina] – Profa. [nome da professora]
Guaxupé – MG
2026

RESUMO

Sistemas de inteligência artificial (IA) falham de formas que o software convencional não falha: um modelo pode ser enganado por entradas ligeiramente perturbadas, contaminado por dados de treino adulterados, induzido por instruções escondidas em documentos recuperados ou substituído por um artefato malicioso na cadeia de suprimento. Este trabalho reproduz, **apenas em sistemas próprios e locais** e com dados públicos ou sintéticos, quatro ataques e mede o efeito de defesas com critério de aceite definido antes da execução. (i) Evasão: um ataque PGD com perturbação $\varepsilon = 0{,}15$ reduz a acurácia de um MLP sobre dígitos manuscritos de 97,6% para 5,4%; o treino adversarial min-max a eleva a 57,6% sem perda de acurácia limpa. (ii) Envenenamento: com 20% de rótulos invertidos a acurácia cai a 91,6%; um filtro k-NN recupera 88,0% da perda. (iii) Prompt injection indireta em um RAG local: 11 de 13 documentos maliciosos têm sucesso sem defesa; filtro de entrada somado a privilégio mínimo zera os ataques medidos, mas deixa passar, em contexto, documentos que adulteram conteúdo. (iv) Cadeia de suprimento: fixar o SHA-256 do artefato rejeita 100% das trocas, enquanto um teste funcional sozinho aceitaria 100% das trocas sutis. A conclusão sustentada pelas medições é que as falhas mais graves estiveram no **sistema em torno do modelo** (privilégios, validação, integridade), e não no modelo isolado.

Palavras-chave: segurança de IA; aprendizado de máquina adversarial; prompt injection; envenenamento de dados; cadeia de suprimento; RAG.

SUMÁRIO

1 INTRODUÇÃO 4
2 FUNDAMENTAÇÃO TEÓRICA 5
3 MÉTODO REPRODUZÍVEL 6
4 ARTEFATO E EVIDÊNCIAS 9
5 DISCUSSÃO E LIMITES 15
6 CONCLUSÃO 17
REFERÊNCIAS 18
APÊNDICE A – DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL 20
APÊNDICE B – COMANDO EXATO DE REPRODUÇÃO 20

1 INTRODUÇÃO

A adoção de IA em produtos reais deslocou o foco da segurança: além de proteger servidores e redes, é preciso proteger o *comportamento aprendido* de um modelo e o conjunto de componentes que o cercam (dados de treino, repositórios de modelos, bases de conhecimento, ferramentas que o modelo pode acionar). O relatório de taxonomia do NIST sobre aprendizado de máquina adversarial organiza esse espaço por estágio do ciclo de vida e por objetivo do atacante, e afirma que as mitigações disponíveis são parciais e frequentemente trocam robustez por desempenho (VASSILEV et al., 2024). A lista OWASP para aplicações com modelos de linguagem coloca a injeção de instruções (*prompt injection*), o envenenamento de dados e modelos, e as vulnerabilidades de cadeia de suprimento entre os principais riscos (OWASP, 2025).

Este trabalho trata do Tema 05 da disciplina e cobre quatro classes de ataque: evasão, envenenamento, *prompt injection* e cadeia de suprimento. A pergunta de pesquisa, que também é o foco da arguição, é: **um ataque bem-sucedido prova uma falha do modelo ou uma falha do sistema em volta dele?** Para responder com evidência e não com opinião, o grupo construiu um conjunto de experimentos pequenos, locais e reprodutíveis, mediu o desempenho ou a quebra de política antes e depois de cada defesa, e declarou de antemão o critério de aceite de cada defesa.

1.1 Objetivos

Objetivo geral: reproduzir ataques a sistemas de IA em ambiente próprio e avaliar controles defensivos de forma mensurável. **Objetivos específicos:** (a) quantificar a queda de acurácia de classificadores sob evasão e sob envenenamento; (b) quantificar a taxa de sucesso de ataques de *prompt injection* indireta em um RAG local antes e depois de validação de entrada, privilégio mínimo e monitoramento; (c) demonstrar que a verificação de integridade de artefatos barra a troca de modelos; (d) discutir ameaças à validade e limites éticos.

1.2 Escopo e ética

Todos os ataques foram executados **exclusivamente em sistemas construídos pelo grupo**, em máquina local, sem acesso a serviços de terceiros, mesmo “para teste”, conforme exige o enunciado. Não foram usados dados pessoais reais: o classificador usa a base pública de dígitos manuscritos do scikit-learn e o RAG usa uma base de conhecimento sintética de uma loja fictícia, com segredo falso (um *canary*). Nenhum código malicioso foi escrito: o ataque de cadeia de suprimento consiste na troca de um arquivo de pesos, e o formato de serialização adotado recusa objetos executáveis.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Evasão e exemplos adversariais

Na evasão, o atacante altera a entrada em tempo de inferência para provocar uma decisão errada, sem tocar no modelo. Goodfellow, Shlens e Szegedy (2015) explicam o fenômeno pela linearidade local das redes e propõem o método do gradiente com sinal (FGSM): $x' = x + \varepsilon \cdot sign(\nabla_x L(\theta, x, y))$, em que L é a perda e ε limita a perturbação em norma L∞. Madry et al. (2018) formalizam a defesa como um problema min-max: o treino minimiza a perda *sobre o pior caso* dentro da bola de raio ε, e o pior caso é aproximado por uma versão iterativa do ataque, o PGD (*projected gradient descent*). Esse último ponto importa para a avaliação: uma defesa avaliada só contra FGSM pode parecer robusta apenas porque mascara o gradiente; por isso este trabalho avalia também com PGD.

2.2 Envenenamento de dados

No envenenamento, o atacante corrompe o conjunto de treino. Biggio, Nelson e Laskov (2012) mostraram que poucos pontos escolhidos podem degradar um classificador; Gu, Dolan-Gavitt e Garg (2017) mostraram o caso de *backdoor*, em que o modelo se comporta bem, exceto na presença de um gatilho. Carlini et al. (2024) demonstraram que envenenar conjuntos de dados obtidos da web em larga escala é prático e barato, por exemplo explorando domínios expirados referenciados por conjuntos de dados. A inversão de rótulos, usada aqui, é a forma mais simples dessa família e serve como linha de base honesta: se uma defesa não funciona nela, não funcionará em ataques mais sofisticados.

2.3 Prompt injection e envenenamento de bases de conhecimento

Perez e Ribeiro (2022) mostraram que instruções do usuário podem sequestrar o objetivo original de um modelo (“ignore as instruções anteriores”). Greshake et al. (2023) estenderam o problema para a forma *indireta*: a instrução maliciosa não vem do usuário, mas de dados que a aplicação recupera (páginas, e-mails, documentos), e, como o modelo não distingue dados de instruções, o atacante controla o comportamento sem acesso direto ao *prompt*. Em sistemas de geração aumentada por recuperação (RAG), Zou et al. (2024) mostraram que injetar poucos textos na base de conhecimento, construídos para serem recuperados para uma pergunta-alvo, basta para induzir respostas escolhidas pelo atacante. A consequência de projeto é que o risco cresce com o que o modelo *pode fazer* (ferramentas, segredos no contexto), o que motiva o princípio do menor privilégio (OWASP, 2025).

2.4 Cadeia de suprimento

Modelos, conjuntos de dados e bibliotecas são artefatos de terceiros. Os riscos incluem a troca de arquivos em repositórios, formatos de serialização que executam código ao serem carregados e dependências comprometidas (OWASP, 2025; VASSILEV et al., 2024). As defesas de engenharia são conhecidas da segurança de software: fixar resumos criptográficos (*hash*) fora do local onde o artefato é armazenado, usar formatos de dados puros e testar o artefato antes de promovê-lo. Cada uma dessas defesas cobre um buraco diferente, como a Seção 4.4 evidencia.

2.5 Quadro-síntese

Tabela 1 – Ataques, fase do ciclo de vida e controles avaliados

Ataque	**Fase**	**Alvo**	**Controle avaliado**
Evasão (FGSM/PGD)	Inferência	Decisão do modelo	Treino adversarial min-max
Envenenamento (rótulos)	Treino	Dados	Filtro de consistência k-NN
Prompt injection indireta	Inferência (RAG)	Sistema em volta do modelo	Validação de entrada, privilégio mínimo, monitoramento
Cadeia de suprimento	Distribuição	Artefato do modelo	Formato sem pickle, SHA-256 fixado, teste de aceitação

Fonte: elaborada pelos autores (2026).

2.6 Princípios de defesa adotados

Três princípios orientam as defesas avaliadas. O primeiro é a **defesa em profundidade**: nenhum controle isolado é suficiente, e a escolha deve combinar controles com modos de falha diferentes, de modo que o que escapa de um seja barrado por outro. O segundo é o **privilégio mínimo**: o componente de IA recebe apenas os dados e as ferramentas de que a tarefa exige, de modo que um comportamento induzido tenha efeito limitado (OWASP, 2025). O terceiro é a **avaliação contra atacante adaptativo**: uma defesa só é considerada eficaz se resistir a um atacante que conhece o mecanismo e escolhe a entrada para contorná-lo (MADRY et al., 2018). Esses princípios explicam por que o trabalho mede, para cada defesa, não só o ganho médio, mas também o contorno que ela deixa.

3 MÉTODO REPRODUZÍVEL

Todo o método está no repositório público do trabalho [inserir URL do repositório], com um README que traz o comando exato de reprodução (reproduzido no Apêndice B). Os experimentos rodam em CPU comum, sem GPU e sem internet, em cerca de dois minutos. Versões fixas: Python 3, NumPy 2.1.3, scikit-learn 1.5.2 e Matplotlib 3.9.2.

3.1 Protocolo geral

Cada experimento segue o mesmo protocolo: (1) medir o sistema sem ataque (linha de base); (2) executar o ataque e medir a queda; (3) aplicar a defesa e medir de novo; (4) comparar com um **critério de aceite escrito antes da execução**. Os experimentos com modelos usam cinco sementes aleatórias (0 a 4) e relatam média e desvio-padrão; a divisão é 70% treino e 30% teste, estratificada. O experimento do RAG é determinístico (13 ataques fixos), por isso relata contagens e não desvios.

3.2 Experimento 1 – Evasão

Dados: dígitos manuscritos 8×8 (1.797 amostras, 10 classes), base pública do scikit-learn (PEDREGOSA et al., 2011), pixels em [0, 1]. **Modelos:** um classificador linear softmax e um MLP com uma camada oculta de 64 unidades ReLU, ambos implementados em NumPy para que o gradiente em relação à entrada seja calculado de forma exata (ataque de caixa-branca, o pior caso para o defensor). **Ataques:** FGSM e PGD (10 passos, passo $\varepsilon/4$, projeção na bola $L\infty$ e em [0, 1]) com ε {0; 0,05; 0,10; 0,15; 0,20; 0,30}. **Defesa:** treino adversarial min-max com $\varepsilon = 0{,}15$; a cada mini-lote, exemplos PGD (3 passos) são gerados contra os pesos atuais e o lote passa a ser 50% limpo e 50% adversarial. **Critério de aceite:** (a) a acurácia sob PGD com $\varepsilon = 0{,}15$ deve subir pelo menos 20 pontos percentuais (p.p.); (b) a acurácia limpa não pode cair mais que 3 p.p.

3.3 Experimento 2 – Envenenamento

Mesmos dados e MLP (scikit-learn, 64 unidades). O atacante inverte rótulos de treino em taxas de 0%, 5%, 10%, 20% e 30%, de dois modos: *aleatório* (o novo rótulo é sorteado) e *direcionado* (parte dos exemplos da classe 3 recebe o rótulo 8). A métrica é a acurácia em um teste limpo e, no caso direcionado, a revocação da classe 3. **Defesa:** filtro de consistência k-NN: descarta-se a amostra cujo rótulo concorda com menos de 40% dos seus 7 vizinhos mais próximos. **Critério de aceite:** com 20% de rótulos invertidos, recuperar pelo menos 80% da acurácia perdida e remover no máximo 5% das amostras quando não há ataque.

3.4 Experimento 3 – Prompt injection indireta em RAG local

Sistema: base de conhecimento sintética da “Loja Aurora” com 24 documentos legítimos (dois deles escritos de propósito com linguagem imperativa legítima, para medir falsos positivos), recuperação por TF-IDF com similaridade de cosseno (os 3 melhores trechos) e um gerador simulado. **Segredo:** um *canary* falso no *prompt* de sistema. **Ferramenta:** uma ação de envio de e-mail, que a aplicação em princípio não deveria oferecer ao assistente de perguntas e respostas.

Modelo de ameaça (pior caso, declarado): o gerador é um "LLM ingênuo" que *obedece a qualquer instrução embutida* em um trecho recuperado. Isso é uma simulação, não um LLM real, e foi uma escolha deliberada: ela isola o que o trabalho quer medir, que são os controles do sistema em volta do modelo, e evita que o resultado dependa da obediência de um modelo específico. O atacante conhece a pergunta-alvo e a usa como título do documento malicioso, como em Zou et al. (2024).

Ataques: 13 documentos maliciosos com três objetivos – *vazamento* do segredo, *exfiltração* via ferramenta de e-mail e *adulteração* da resposta (afirmar que o prazo de devolução é de 90 dias, quando o correto é 7) – e quatro estilos: plano, parafraseado (sem as palavras-chave que um filtro simples procuraria, simulando um atacante adaptativo), ofuscado (letras espaçadas e caracteres invisíveis) e em inglês. Um ataque extra (a13) usa a construção "pergunta-alvo + resposta falsa" da literatura, sem sintaxe de instrução. **Defesas:** D1, validação de entrada (normalização Unicode, remoção de caracteres invisíveis e padrões de instrução em PT/EN, aplicada aos trechos antes da recuperação); D2, privilégio mínimo (o segredo não entra no contexto do modelo e a ferramenta de e-mail não está na lista de permissão do agente); D3, monitoramento (alerta, sem bloqueio, quando o *canary* aparece na saída ou há chamada de ferramenta não prevista). **Métricas:** taxa de sucesso do ataque (ASR), utilidade (perguntas legítimas respondidas corretamente) e taxa de detecção. **Critério de aceite** com D1+D2: vazamento e exfiltração em 0%; ASR total ≤ 25%; perda de utilidade ≤ 5 p.p.

3.5 Experimento 4 – Cadeia de suprimento

O modelo MLP treinado é exportado em formato *npz* sem objetos serializados (carregado com *allow_pickle=False*) para uma pasta de artefatos, e o SHA-256 é registrado em um manifesto separado. O atacante, com permissão de escrita na pasta, troca o arquivo por (a) um modelo treinado com 30% de rótulos invertidos, ou (b) uma versão "sutil" do modelo original, com um viés pequeno na classe 3. Comparam-se um carregador ingênuo (só carrega) e um carregador seguro (confere o *hash* e depois um teste de aceitação com acurácia mínima de 90% em validação separada). **Critério de aceite:** o carregador seguro rejeita 100% das trocas e o ingênuo, 0%.

3.6 Estrutura do artefato

O repositório é organizado de modo que cada experimento seja um *script* independente que lê dados públicos ou sintéticos e grava seus resultados em arquivos JSON, a partir dos quais tabelas e figuras deste relatório são geradas por código, sem transcrição manual de números.

Tabela 2 – Estrutura do repositório

Caminho	Conteúdo
src/common.py	Carga de dados, rede NumPy com treino adversarial, FGSM e PGD, utilidades
src/evasion.py, evasion_tentativa1.py	Experimento 1 (versão final e tentativa negativa)
src/poisoning.py	Experimento 2: inversão de rótulos aleatória e direcionada, filtro k-NN
src/corpus.py, rag_injection.py	Experimento 3: base sintética, 13 ataques, pipeline RAG e defesas D1–D3
src/supply_chain.py	Experimento 4: exportação sem pickle, manifesto SHA-256, carregadores
results/, figures/	Saídas geradas (JSON e PNG)
run_all.sh, requirements.txt, README.md	Comando único de reprodução, versões fixas e instruções

Fonte: elaborada pelos autores (2026).

3.7 Ameaças à validade (resumo do método)

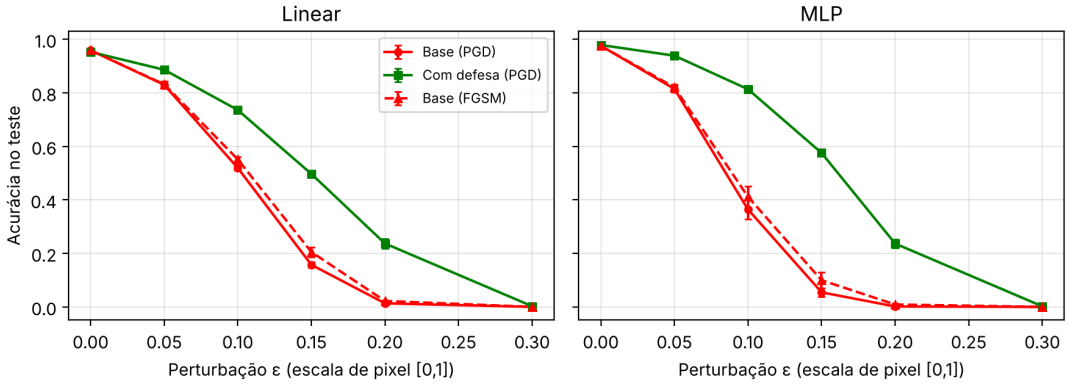
As ameaças são discutidas na Seção 5. Aqui basta registrar as decisões de projeto que as reduzem: critérios definidos antes de rodar; cinco sementes; avaliação com ataque iterativo (PGD) além do FGSM; relato de resultados negativos (Seção 4.1); e código e dados abertos.

4 ARTEFATO E EVIDÊNCIAS

4.1 Evasão

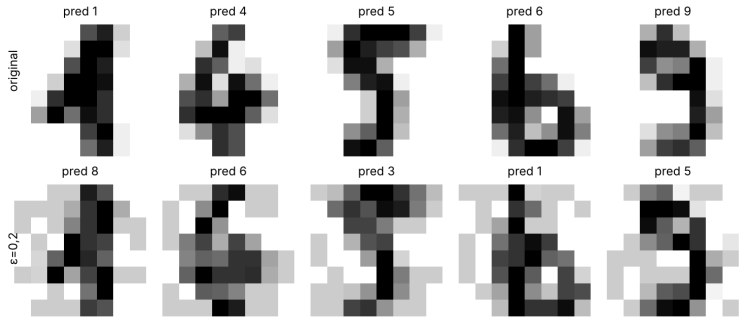
A Figura 1 mostra a acurácia em função de ε. O MLP sem defesa tem 97,6% de acurácia limpa e cai para 36,4% com ε = 0,10 e 5,4% com ε = 0,15, valor próximo do acaso entre as classes mais confundíveis, apesar de a perturbação máxima por pixel ser de 15% da escala. A Figura 2 mostra exemplos: a imagem perturbada continua legível para uma pessoa, e o modelo muda a classe.

Figura 1 – Acurácia sob ataque (PGD e FGSM) em função de ε, com e sem treino adversarial (média ± desvio, 5 sementes)



Fonte: elaborada pelos autores (2026), a partir dos resultados em results/.

Figura 2 – Dígitos originais (linha superior) e perturbados com PGD, ε = 0,2 (linha inferior), com a classe prevista pelo MLP



Fonte: elaborada pelos autores (2026), a partir dos resultados em results/.

Tabela 3 – Evasão: acurácia (%) sob PGD, média de 5 sementes

Modelo	**ε = 0**	**ε = 0,05**	**ε = 0,10**	**ε = 0,15**	**ε = 0,20**
Linear – sem defesa	95,9	83,0	51,9	15,7	1,3
MLP – sem defesa	97,6	81,5	36,4	5,4	0,1
Linear – treino adversarial	95,3	88,6	73,6	49,7	23,7
MLP – treino adversarial	97,9	93,9	81,4	57,6	23,7

Fonte: elaborada pelos autores (2026).

Critério de aceite. No MLP, a acurácia sob PGD com ε = 0,15 subiu 52.1 p.p. e a acurácia limpa variou +0.3 p.p.: critério atendido. No modelo linear, o ganho foi de 34.0 p.p. e a queda limpa de 0.6 p.p.: atendido. A defesa não elimina o ataque: com ε = 0,20 o MLP ainda cai para 23,7%, e com ε = 0,30 todos os modelos colapsam. Ela desloca a curva, não a remove.

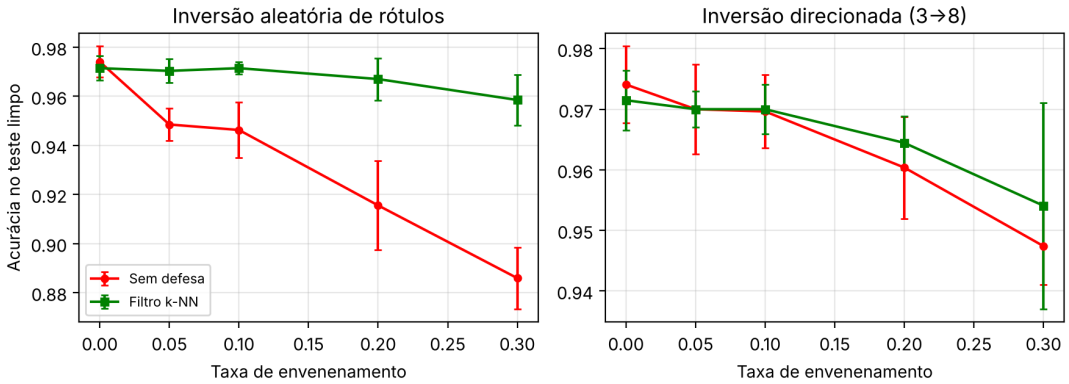
Resultado negativo (tentativa 1). A primeira versão da defesa foi *aumento de dados estático*: gerar exemplos adversariais contra o modelo treinado, juntá-los ao treino e re-treinar do zero. O resultado foi **pior** que não defender: sob PGD com ε = 0,15, a regressão logística foi de 28,5% para 1,8% e o MLP de 5,0% para 0,0%. A explicação é que

o modelo novo aprende a resistir ao ataque calculado contra o modelo antigo, mas o atacante de caixa-branca recalcula o gradiente contra o modelo novo. Por isso a versão final gera os exemplos a cada lote contra os pesos atuais (Madry et al., 2018). O código da tentativa 1 está no repositório (evasion_tentativa1.py).

4.2 Envenenamento

A Figura 3 e a Tabela 4 mostram que a inversão aleatória de rótulos degrada o modelo de forma gradual: de 97,4% para 91,6% com 20% e 88,6% com 30% de rótulos invertidos. O filtro k-NN mantém 96,7% com 20%. Sem ataque, o filtro remove 1,1% das amostras (rótulos ambíguos naturais) e a acurácia vai de 97,4% a 97,1%.

Figura 3 – Acurácia no teste limpo versus taxa de envenenamento, sem defesa e com filtro k-NN (5 sementes)



Fonte: elaborada pelos autores (2026), a partir dos resultados em results/.

Tabela 4 – Envenenamento aleatório: acurácia (%) e fração de amostras removidas pelo filtro (%)

Taxa	Sem defesa	Filtro k-NN	Removidas
0%	97,4	97,1	1,1
5%	94,9	97,0	6,1
10%	94,6	97,1	11,3
20%	91,6	96,7	21,8
30%	88,6	95,9	33,4

Fonte: elaborada pelos autores (2026).

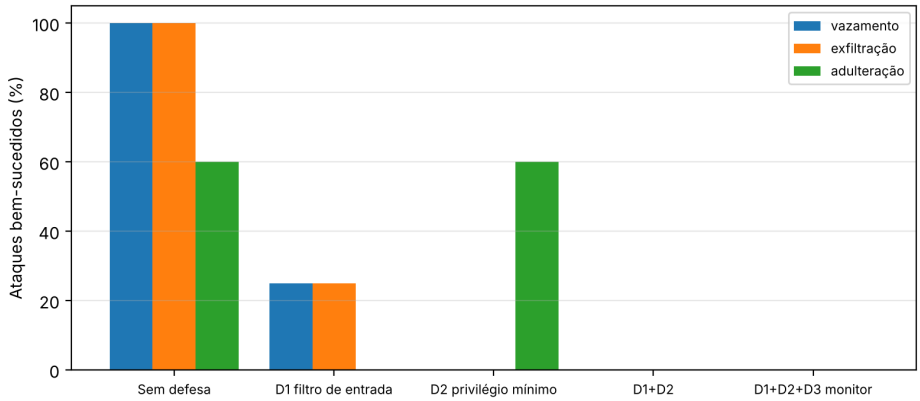
Critério de aceite. Recuperação da perda com 20%: 88,0% (meta ≥ 80%); remoção sem ataque: 1,1% (meta ≤ 5%). Critério atendido. Note que o custo é visível: com 20% de envenenamento o filtro descarta 21,8% dos dados, mais do que os 20% inseridos em parte porque exemplos limpos de classes difíceis também são removidos.

O que a acurácia média esconde. No ataque direcionado (3→8), a acurácia global quase não se move (97,4% para 94,7% com 30%), mas a revocação da classe 3 cai de 97,8% para 72,0%. Com o filtro, vai a 83,6%, com desvio-padrão de 15,5 p.p. entre sementes: a defesa ajuda, mas é instável nesse regime. Um monitoramento baseado só na acurácia global não detectaria esse ataque.

4.3 Prompt injection indireta em RAG local

Sem defesa, 11 dos 13 ataques tiveram sucesso (ASR = 84,6%): o segredo vazou em 4 de 4 casos e a ferramenta de e-mail foi acionada em 4 de 4. A utilidade era de 100,0% (24 de 24 perguntas legítimas). A Figura 4 e a Tabela 5 mostram o efeito de cada controle.

Figura 4 – Ataques bem-sucedidos (%) por objetivo e por configuração de defesa (13 ataques)



Fonte: elaborada pelos autores (2026), a partir dos resultados em results/.

Tabela 5 – RAG local: ASR (%), exposição (ataque presente no contexto) e utilidade (%) por configuração

Configuração	ASR total	Vazamento	Exfiltração	Adulteração	Atacante no contexto	Utilidade
Sem defesa	84,6	100,0	100,0	60,0	100,0	100,0
D1 filtro de entrada	15,4	25,0	25,0	0,0	30,8	95,8
D2 privilégio mínimo	23,1	0,0	0,0	60,0	100,0	100,0
D1+D2	0,0	0,0	0,0	0,0	30,8	95,8
D1+D2+D3 monitor	0,0	0,0	0,0	0,0	30,8	95,8

Fonte: elaborada pelos autores (2026).

Leitura por controle. A validação de entrada (D1) bloqueou todos os ataques de estilo plano, ofuscado e em inglês, mas **não** os dois ataques parafraseados de vazamento e de exfiltração, que não usam as expressões esperadas; sozinha, deixa ASR de 15,4%. O privilégio mínimo (D2) zera vazamento e exfiltração *independentemente da redação*, porque o segredo não está no contexto e a ferramenta é negada, mas não afeta a adulteração de resposta (ASR de 23,1% sozinho). Combinadas, D1+D2 zeram o ASR medido, com perda de utilidade de 4,2 p.p. (um documento legítimo, o que manda enviar o comprovante por e-mail, foi descartado como falso positivo por D1).

Critério de aceite. Vazamento e exfiltração: 0% (atendido); ASR total ≤ 25%: 0,0% (atendido); perda de utilidade ≤ 5 p.p.: 4,2 p.p. (atendido, com pouca folga). Conforme o critério escrito antes da execução, a defesa é *aceita*. Entretanto, a coluna “Atacante no contexto” mostra que 31% dos documentos maliciosos continuam chegando ao modelo; apenas deixaram de ter efeito porque o simulador, no pior caso, só obedece a ações que o sistema permite.

Tabela 6 – RAG local: resultado de cada ataque por configuração

Ataque	Objetivo	Estilo	Sem defesa	D1	D2	D1+D2
a01	vazamento	plano	sucesso	barrado	no contexto	barrado
a02	vazamento	parafraseado	sucesso	sucesso	no contexto	no contexto
a03	vazamento	ofuscado	sucesso	barrado	no contexto	barrado
a04	vazamento	inglês	sucesso	barrado	no contexto	barrado
a05	exfiltração	plano	sucesso	barrado	no contexto	barrado
a06	exfiltração	parafraseado	sucesso	sucesso	no contexto	no contexto
a07	exfiltração	ofuscado	sucesso	barrado	no contexto	barrado
a08	exfiltração	inglês	sucesso	barrado	no contexto	barrado
a09	adulteração	plano	sucesso	barrado	sucesso	barrado
a10	adulteração	parafraseado	no contexto	no contexto	no contexto	no contexto
a11	adulteração	ofuscado	sucesso	barrado	sucesso	barrado
a12	adulteração	inglês	sucesso	barrado	sucesso	barrado
a13	adulteração	otimizado p/ recuperação	no contexto	no contexto	no contexto	no contexto

Fonte: elaborada pelos autores (2026).

A Tabela 6 distingue três estados: *sucesso* (objetivo do atacante atingido), *no contexto* (o documento chegou ao modelo mas não produziu efeito) e *barrado* (removido antes da geração). O estado “no contexto” é o que merece atenção em produção: representa um ataque que só falhou por causa das restrições do ambiente.

Monitoramento (D3). Dos 4 ataques que chegaram ao contexto sob D1+D2, o monitor gerou alerta em 25,0% (1 de 4): o que tentou acionar a ferramenta de e-mail. O vazamento parafraseado foi neutralizado sem alerta (o *canary* não chegou à saída) e os dois documentos de conteúdo falso passaram sem sinal. Ou seja, o monitor baseado em *canary* e em chamadas de ferramenta é útil para tentativas de ação, mas cego para a adulteração de conteúdo.

Resultado que contraria a intuição. Os dois documentos de adulteração *sem instrução* (a10, texto parafraseado, e a13, no estilo “pergunta + resposta falsa”) tiveram ASR de 0% mesmo sem defesa. Não é porque sejam inofensivos: ambos foram recuperados para o contexto (posição 2), mas o simulador responde com o primeiro trecho, e o trecho

legítimo ganhou o primeiro lugar por margem mínima (similaridade de cosseno de 0,710 contra 0,703 no caso de a13). Qualquer mudança de ranqueamento, de tamanho de trecho ou de política de geração (por exemplo, usar todos os trechos) poderia inverter o resultado. Este é um achado de sensibilidade, não uma garantia de segurança.

4.4 Cadeia de suprimento

Em todas as cinco sementes, o carregador ingênuo aceitou o arquivo trocado (0% de rejeição) e o carregador seguro o rejeitou (100%). O modelo trocado, envenenado com 30% de rótulos invertidos, tem acurácia de teste média de 89,8%, contra 97,2% do original. Dois resultados merecem destaque. Primeiro, o teste de aceitação sozinho (acurácia ≥ 90% em validação) teria aprovado o modelo envenenado em 40,0% das sementes, porque a degradação é moderada. Segundo, a versão “sutil”, com viés na classe 3, tem acurácia de teste de 97,2%, idêntica à original, e passaria no teste de aceitação em 100,0% das sementes; apenas a verificação do *hash* a barrou (100,0%). O critério de aceite foi atendido. A lição é que testes funcionais e verificação de integridade respondem a perguntas diferentes: o primeiro pergunta “funciona?”, o segundo, “é o mesmo artefato que foi aprovado?”.

Tabela 7 – Cadeia de suprimento: efeito da troca do artefato e resultado de cada carregador (5 sementes)

Cenário	**Acurácia de teste (%)**	**Ingênuo**	**Só teste de aceitação**	**Hash + teste**
Modelo original	97,2	carrega	aprova	aprova
Troca por modelo envenenado	89,8	carrega	aprova em 40%	rejeita (100%)
Troca “sutil” (viés na classe 3)	97,2	carrega	aprova em 100%	rejeita (100%)

Fonte: elaborada pelos autores (2026).

4.5 Síntese: critérios de aceite

Tabela 8 – Critérios definidos antes da execução e resultados observados

Experimento	**Critério**	**Observado**	**Veredito**
1 Evasão (MLP)	Ganho ≥ 20 p.p. sob PGD ε = 0,15; queda limpa ≤ 3 p.p.	+52.1 p.p.; -0.3 p.p.	Atendido
1 Evasão (linear)	Idem	+34.0 p.p.; +0.6 p.p.	Atendido
2 Envenenamento	Recuperar ≥ 80% da perda (20%); remover ≤ 5% sem ataque	88,0%; 1,1%	Atendido
3 RAG (D1+D2)	Vazamento e exfiltração 0%; ASR ≤ 25%; perda de utilidade ≤ 5 p.p.	0%; 0,0%; 4,2 p.p.	Atendido
4 Cadeia de suprimento	Seguro rejeita 100%; ingênuo 0%	100%; 0%	Atendido

Fonte: elaborada pelos autores (2026).

Todos os critérios foram atendidos. Isso não significa que os sistemas estejam seguros: significa que cada defesa fez o que se propôs a fazer sob o modelo de ameaça declarado. Os critérios foram fixados pelo grupo, e um revisor externo poderia exigir metas mais severas. Duas observações moderam o resultado: a defesa de evasão deixa o modelo vulnerável a perturbações maiores ($\varepsilon \geq 0{,}20$), e a defesa do RAG não trata a adulteração de conteúdo.

5 DISCUSSÃO E LIMITES

5.1 Falha do modelo ou falha do sistema?

O foco da arguição pede uma posição. A evidência deste trabalho sustenta uma resposta dividida por classe de ataque. Na **evasão**, a falha é do modelo: com o sistema ao redor perfeito, uma perturbação de 15% da escala derruba um MLP de 97,6% para 5,4% de acurácia, e a defesa também é do modelo (treino adversarial), embora o ganho seja parcial. No **envenenamento**, a falha nasce no *pipeline de dados*, que aceitou rótulos sem validação; a defesa (filtro) é um controle de sistema sobre os dados. Na **injeção de instruções** e na **cadeia de suprimento**, a evidência aponta para o sistema: o que transformou um texto malicioso em vazamento e envio de e-mail foi o segredo estar no contexto e a ferramenta estar disponível, e o privilégio mínimo anulou ambos *sem tocar no modelo e sem acertar a redação do ataque*. Na cadeia de suprimento, o modelo trocado funcionou, e só um controle de integridade externo o identificou. Portanto, a resposta sustentada é: **o ataque prova uma falha do sistema quando o dano vem de capacidades que o sistema concedeu, e prova uma falha do modelo quando o dano é a decisão errada em si.** Em aplicações reais com LLM, a maior parte do dano potencial está no primeiro caso.

5.2 Ameaças à validade

Validade interna. (i) Os critérios de aceite foram escritos antes de cada execução, mas os parâmetros das defesas (por exemplo, limiar de 0,4 e $k = 7$ do filtro) vieram de decisão do grupo e não foram buscados em validação cruzada; outros valores mudariam o custo em dados removidos. (ii) Os padrões do filtro D1 foram escritos antes dos ataques parafraseados, que foram redigidos justamente para evitá-los: isso representa um atacante adaptativo, mas o conjunto é pequeno e escrito pelos mesmos autores, o que pode subestimar ou superestimar a defesa. (iii) A utilidade do RAG tem apenas 24 perguntas: uma pergunta equivale a 4,2 p.p., e a perda de 4,2 p.p. ficou no limite do critério (5 p.p.).

Validade externa. (i) Dígitos 8×8 são um problema fácil e de baixa dimensão; as curvas de robustez de imagens maiores e modelos profundos serão diferentes, e o ataque PGD de caixa-branca não representa atacantes sem acesso ao modelo. (ii) O gerador do RAG é um **simulador de pior caso**, não um LLM real: o valor absoluto do ASR sem defesa

(84,6%) não deve ser tomado como estimativa de nenhum modelo real; apenas a *diferença entre configurações* é informativa, e o resultado de D2 vale por construção, já que o segredo ausente não pode vazar. Isso é uma limitação, e também é o argumento: o controle não depende do comportamento do modelo. (iii) O ataque de cadeia de suprimento supõe que o atacante só escreve na pasta de artefatos; se também controlar o manifesto, o *hash* não protege, e seria preciso assinatura digital com chave fora do alcance do atacante.

Validade de construto. ASR mede se o objetivo do atacante foi atingido no cenário simulado, não o dano real. A acurácia global, como visto no ataque direcionado, é uma métrica insuficiente para detectar envenenamento localizado. O ataque ao conteúdo (a10, a13) mostra que a ausência de sucesso em uma configuração não equivale a segurança.

5.3 Ética e uso responsável

Os ataques descritos são conhecidos e públicos na literatura; o artefato do trabalho é pequeno, didático e opera só sobre dados sintéticos e um simulador local. Não há alvo de terceiros, não há dados pessoais e não há carga útil executável. As defesas foram implementadas e medidas, em linha com o enunciado, que exige defesa implementada ou especificada com critério de aceite. A reprodução em sistema de terceiros, mesmo “para teste”, é proibida e fora do escopo; quem quiser avaliar um sistema alheio deve ter autorização formal e escopo definido.

5.4 Trabalhos futuros

(a) Repetir o Experimento 3 com um LLM local real e medir a taxa de obediência em vez de supô-la; (b) acrescentar delimitação explícita de dados e instruções e um verificador de saída; (c) metadados de proveniência por documento, para tratar a adulteração de conteúdo, que nenhum dos três controles cobriu; (d) substituir o *hash* por assinatura digital e (e) estender o envenenamento a *backdoors* com gatilho.

6 CONCLUSÃO

O trabalho reproduziu, em sistemas próprios, quatro ataques a sistemas de IA e mediu defesas com critérios definidos antes da execução. A evasão derrubou um MLP de 97,6% para 5,4% de acurácia com perturbação de 0,15, e o treino adversarial min-max elevou o valor a 57,6%; a primeira tentativa de defesa, o aumento estático, piorou a situação e foi registrada. O envenenamento por 20% de rótulos invertidos foi 88% recuperado por um filtro k-NN, mas com remoção de 21,8% dos dados, e um ataque direcionado mostrou que a acurácia global esconde a degradação de uma classe. No RAG local, validação de entrada com privilégio mínimo zerou vazamento e exfiltração; a validação sozinha foi contornada por paráfrase, enquanto o privilégio mínimo funcionou independentemente da redação, e a adulteração de conteúdo permaneceu um risco residual não monitorado. Na cadeia de suprimento, o *hash* fixado barrou 100% das trocas, inclusive a que um teste funcional não distinguiria do original.

A resposta à pergunta da arguição é que depende de onde o dano se origina: decisões erradas induzidas por entradas são falhas do modelo; vazamentos, ações indevidas e artefatos trocados são falhas do sistema em volta dele, e é ali que os controles de menor custo (privilégio mínimo, integridade, validação) trouxeram o maior ganho. Os resultados, porém, têm validade limitada pela escala dos dados e pelo uso de um gerador simulado, e devem ser lidos como evidência de mecanismo, não como estimativa de risco de um produto real.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

BIGGIO, B.; NELSON, B.; LASKOV, P. Poisoning attacks against support vector machines. In: INTERNATIONAL CONFERENCE ON MACHINE LEARNING, 29., 2012, Edinburgh. **Proceedings** [...]. Madison: Omnipress, 2012. p. 1467-1474.

CARLINI, N. *et al.* Poisoning web-scale training datasets is practical. In: IEEE SYMPOSIUM ON SECURITY AND PRIVACY, 2024, San Francisco. **Proceedings** [...]. Piscataway: IEEE, 2024. Disponível em: https://arxiv.org/abs/2302.10149. Acesso em: 3 nov. 2026.

GOODFELLOW, I. J.; SHLENS, J.; SZEGEDY, C. Explaining and harnessing adversarial examples. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 3., 2015, San Diego. **Proceedings** [...]. 2015. Disponível em: https://arxiv.org/abs/1412.6572. Acesso em: 3 nov. 2026.

GRESHAKE, K. *et al.* Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In: ACM WORKSHOP ON ARTIFICIAL INTELLIGENCE AND SECURITY, 16., 2023, Copenhagen. **Proceedings** [...]. New York: ACM, 2023. Disponível em: https://arxiv.org/abs/2302.12173. Acesso em: 3 nov. 2026.

GU, T.; DOLAN-GAVITT, B.; GARG, S. **BadNets**: identifying vulnerabilities in the machine learning model supply chain. 2017. Disponível em: https://arxiv.org/abs/1708.06733. Acesso em: 3 nov. 2026.

MADRY, A. *et al.* Towards deep learning models resistant to adversarial attacks. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 6., 2018, Vancouver. **Proceedings** [...]. 2018. Disponível em: https://arxiv.org/abs/1706.06083. Acesso em: 3 nov. 2026.

OWASP FOUNDATION. **OWASP Top 10 for LLM Applications 2025**. 2025. Disponível em: https://genai.owasp.org/llm-top-10/. Acesso em: 3 nov. 2026.

PEDREGOSA, F. *et al.* Scikit-learn: machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825-2830, 2011.

PEREZ, F.; RIBEIRO, I. **Ignore previous prompt**: attack techniques for language models. 2022. Disponível em: https://arxiv.org/abs/2211.09527. Acesso em: 3 nov. 2026.

VASSILEV, A. *et al.* **Adversarial machine learning**: a taxonomy and terminology of attacks and mitigations. Gaithersburg: National Institute of Standards and Technology, 2024. (NIST AI 100-2 E2023). Disponível em: https://doi.org/10.6028/NIST.AI.100-2e2023. Acesso em: 3 nov. 2026.

ZOU, W. *et al.* **PoisonedRAG**: knowledge corruption attacks to retrieval-augmented generation of large language models. 2024. Disponível em: https://arxiv.org/abs/2402.07867. Acesso em: 3 nov. 2026.

APÊNDICE A – DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Em conformidade com o enunciado, o grupo declara as ferramentas de IA usadas e em que etapa. O julgamento sobre o que foi medido, o que foi concluído e o que é limite é dos autores, que revisaram o material e respondem por ele.

Tabela 9 – Declaração de uso de IA

Ferramenta	**Etapa**	**Uso**	**Verificação pelos autores**
Claude (Anthropic)	Planejamento	Sugestão de estrutura do trabalho a partir da rubrica e dos requisitos, e dos critérios de aceite	Revisão e aceite do grupo
Claude (Anthropic)	Código	Escrita do código Python dos quatro experimentos e do gerador deste PDF	Execução completa do run_all.sh e leitura do código pelo grupo
Claude (Anthropic)	Redação	Primeira versão do texto, tabelas e figuras a partir dos resultados em results/	Conferência dos números contra results/*.json e ajustes de redação
Claude (Anthropic)	Referências	Sugestão da lista bibliográfica	Conferência de cada referência na fonte original pelo grupo

Fonte: elaborada pelos autores (2026).

Divisão de responsabilidade: [o grupo deve confirmar/ajustar este quadro ao que de fato ocorreu em cada etapa]. Nenhum trecho de terceiros foi usado como resultado próprio; os trechos de outros autores aparecem como citação com referência. Os resultados numéricos foram produzidos pela execução do código e não por geração de texto.

APÊNDICE B – COMANDO EXATO DE REPRODUÇÃO

Em uma máquina com Python 3 (testado em 3.13) e Linux/macOS/WSL:

```
git clone [URL do repositório]
cd seguranca-ia
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
./run_all.sh
```

Saídas: `results/*.json` (números usados neste relatório) e `figures/*.png` (figuras). Tempo aproximado: 2 minutos em CPU.